

DA5402 Assignment-2

Author: Anubhav Agarwal, Sayantika Chakraborty

Project: Multi-Modal Orchestrated Cluster

Introduction

In this project, a distributed multi-modal AI microservice is deployed that combines translation, image generation, named entity recognition (NER), and text-to-speech (TTS) in a single application based on FastAPI. The idea was to reproduce a realistic production process of collaborative development, containerization, orchestration, and distributed load balancing as opposed to concentrating on the implementation of AI models alone. Git feature branches were used to create the system with pull-request based integration and purposely introduced merge conflicts to encourage collaborative problems. It was put into a containerized Docker application and deployed on a Docker Swarm cluster consisting of a manager node (HP Victus machine) and a worker node (Docker Desktop environment). Various instances were tested to indicate the distribution of requests over Swarm ingress routing mesh.

Architectural Design

The system follows a modular layered architecture separating API routing, service logic, and deployment infrastructure. All AI capabilities are individually implemented, but offered as a single REST interface.

Core Layers

1. API Layer (FastAPI)
Handles request routing, validation, and response formatting.
2. Service Layer
Implements AI functionality:
 - Translation
 - Image Generation
 - Named Entity Recognition
 - Text-to-Speech
3. Schema Layer
Defines request and response models using Pydantic.
4. Container Layer
Packages the application into Docker images to ensure consistent runtime environments.
5. Orchestration Layer
Docker Swarm manages container replicas, networking, and load balancing.

Each API response includes a container identifier using `socket.gethostname()` to trace which replica processed the request.

Request Flow

Client -> Swarm Ingress -> Load Balancer -> Container Replica -> AI Service -> Response
This architecture mirrors production AI deployment pipelines where modularity and scalability are critical.

Collaborative Development and Conflict Resolution

Development followed a feature-branch workflow:

- feature/translation
- feature/image-generation
- feature/ner
- feature/tts

Pull requests were reviewed before merging into the main branch. A merge conflict was intentionally triggered when multiple branches modified shared files simultaneously. Conflicts also occurred due to accidental inclusion of generated `__pycache__` files. Resolution involved removing generated artifacts, manually merging router imports, and retaining the correct implementations. The documented resolution process demonstrates practical collaborative development challenges.

Containerization

The application was containerized using a lightweight Python base image with dependencies installed from a requirements file. API keys were securely injected through environment variables rather than hardcoding credentials.

Docker layer caching was utilized by copying dependency files before application source code, ensuring dependency installation layers remain cached unless dependencies change. This significantly improves rebuild efficiency.

Each container exposes port 8000 internally and returns its hostname identifier in responses to enable tracking during load testing.

Docker Swarm Deployment

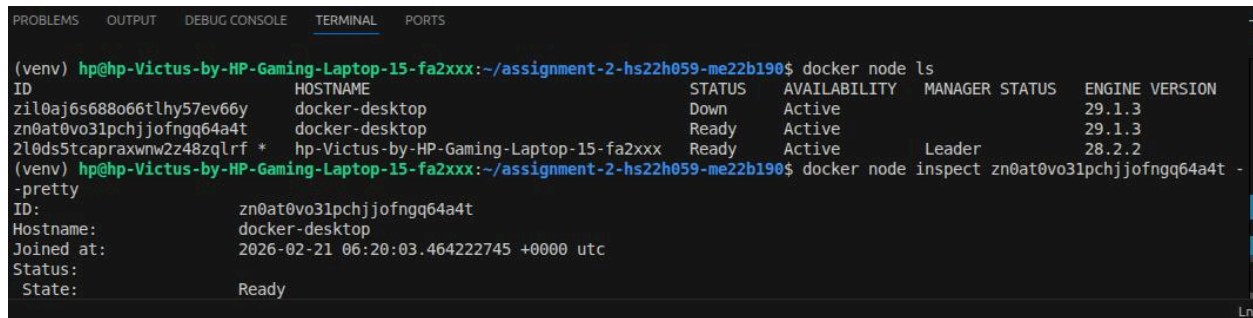
A Docker Swarm cluster was initialized with:

- Manager Node: HP Victus machine (Leader)
- Worker Node: Docker Desktop environment

The service was deployed with four replicas using a stack configuration.

Docker Swarm automatically creates an overlay network, enabling containers across multiple machines to communicate as if they were on the same network. The ingress routing mesh exposes a single endpoint while distributing traffic across replicas using round-robin scheduling.

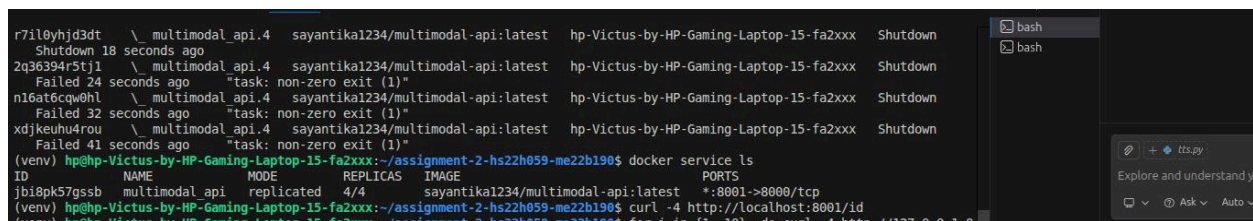
Figure 1: Docker Node Configuration



```
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$ docker node ls
ID                                HOSTNAME            STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
zil0aj6s688o66tlhy57ev66y        docker-desktop      Down      Active           ●                29.1.3
zn0at0vo31pchjjofofngq64a4t        docker-desktop      Ready     Active           ●                29.1.3
2l0ds5tcapraxwnw2z48zqlrf *      hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx    Ready     Active           Leader            28.2.2
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$ docker node inspect zn0at0vo31pchjjofofngq64a4t -
-pretty
ID:                                zn0at0vo31pchjjofofngq64a4t
Hostname:                          docker-desktop
Joined at:                          2026-02-21 06:20:03.464222745 +0000 utc
Status:                             Ready
State:
```

This figure shows the Swarm cluster configuration where one node acts as the manager (leader) and another as the worker. Both nodes are in active state, confirming successful cluster formation.

Figure 2: Service Deployment with Replicas



```
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$ docker service ls
ID                                NAME              MODE                REPLICAS            IMAGE                                  PORTS
jbi8pk57gssb                     multimodal.api    replicated          4/4                 sayantik1234/multimodal-api:latest   *:8001->8000/tcp
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$ curl -4 http://localhost:8001/id
{"id": "zn0at0vo31pchjjofofngq64a4t", "hostname": "docker-desktop", "status": "Ready", "availability": "Active", "manager_status": "Leader", "engine_version": "28.2.2"}
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$ for i in $(seq 1 10); do curl -4 http://127.0.0.1:8001/id; done
```

This figure demonstrates that the service is deployed in replicated mode with four running instances. The presence of multiple task IDs confirms that Docker Swarm successfully created multiple container replicas for load distribution.

Load Balancing Experiment

To validate distributed execution, a concurrent client script was implemented to send 100 API requests. The script collected container identifiers returned in responses and computed request distribution across replicas.

Figure 3: Container ID Responses from API

```
001/id; echo; done
{"container_id":"8c75e2862481"}
{"container_id":"ee351f816b51"}
{"container_id":"91ddf8146908"}
{"container_id":"e58c18c28f73"}
{"container_id":"8c75e2862481"}
{"container_id":"ee351f816b51"}
{"container_id":"91ddf8146908"}
{"container_id":"e58c18c28f73"}
{"container_id":"8c75e2862481"}
{"container_id":"ee351f816b51"}
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$
```

This figure shows repeated API calls returning different container IDs, indicating that requests are being handled by different replicas rather than a single container.

Figure 4: Concurrent Load Testing Results

```
{"container_id":"ee351f816b51"}
{"container_id":"91ddf8146908"}
{"container_id":"e58c18c28f73"}
{"container_id":"8c75e2862481"}
{"container_id":"ee351f816b51"}
{"container_id":"91ddf8146908"}
{"container_id":"e58c18c28f73"}
{"container_id":"8c75e2862481"}
{"container_id":"ee351f816b51"}
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$ python tester.py
Counter({'91ddf8146908': 25, '8c75e2862481': 25, 'e58c18c28f73': 25, 'ee351f816b51': 25})
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/assignment-2-hs22h059-me22b190$
```

The results show approximately uniform request distribution across four container identifiers, confirming correct operation of Docker Swarm's ingress load balancing mechanism.

Observations and Challenges

Several practical challenges were encountered:

- Container restart loops due to dependency or environment configuration issues
- Merge conflicts caused by committing generated files
- Debugging Swarm deployment required inspection of service logs
- Replica scheduling inconsistencies during initial setup

These challenges reflect real-world deployment complexity when transitioning from local development to distributed systems.